

Universidad Técnica Estatal De Quevedo (UTEQ)

Facultad de Ciencias de la Computación

Ingeniería en Software (Rediseño)

Aplicaciones Web

2025-2026 SPA

*Sistema Integral de Gestión, Control y Seguimiento
de Lectura Interna en la Biblioteca Universitaria mediante Códigos QR e
Inteligencia Artificial*

Entrega 1B — Módulo de Autenticación JWT + Acceso a Datos con Spring Data
JPA

Arias Moreira Maybelin Arias CI:1208729788

Romero Mendez Bryam Steven CI:1250897434

Zambrano Moreira Alison Ariana CI:

Dr. Gleiston Cicerón Guerrero Ulloa. Ing.

Domingo 14 de Junio 2026

URL

<https://github.com/a112i3s4o5n6/TA-Proyecto-Fin-de-Curso-Entrega-1B-M-dulo-de-Autenticaci-n-JWT-Acceso-a-Datos-con-Spring-Data-JPA>

1. N

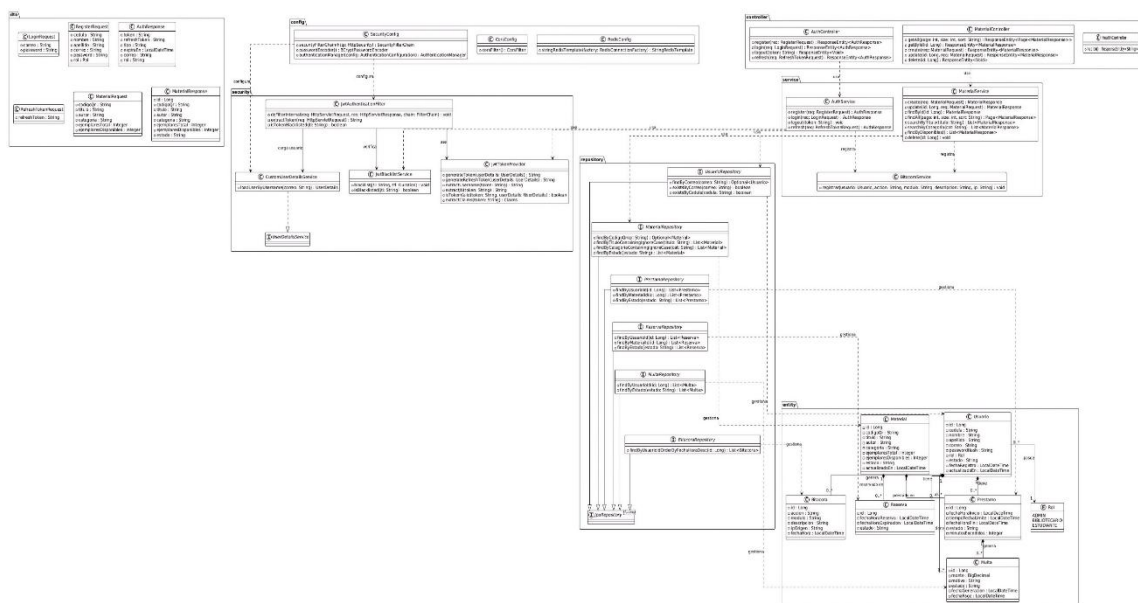
En el presente trabajo se desarrolla el diseño e implementación del sistema de gestión de biblioteca, utilizando Spring Boot como tecnología principal para el backend. El proyecto incluye la organización de las clases y entidades mediante diagramas UML y el modelo entidad-relación de la base de datos, permitiendo una mejor estructura y administración de la información.

Además, se implementó un sistema de autenticación basado en JWT para garantizar la seguridad en el acceso de los usuarios, junto con diferentes controles de seguridad y manejo de permisos según el rol de cada usuario. También se desarrolló el módulo de gestión de materiales con operaciones de registro, consulta, actualización y eliminación de información.

Por otra parte, se realizaron pruebas automatizadas para verificar el correcto funcionamiento del sistema, se creó una colección de solicitudes en Postman para facilitar las pruebas de la API y se analizaron métricas de rendimiento para evaluar la respuesta del sistema. Finalmente, se utilizó Docker para la ejecución de los diferentes servicios y Git para el control de versiones del proyecto, permitiendo una mejor organización y mantenimiento del desarrollo.

2. DIAGRAMA DE CLASES UML REFACTORIZADO

El siguiente diagrama de clases UML **Imagen 1** muestra la estructura del sistema de gestión de biblioteca desarrollado. En él se representan las principales clases del proyecto, sus atributos, métodos y las relaciones existentes entre ellas. La organización de las clases por paquetes permite mantener una mejor estructura del sistema y facilita su desarrollo, mantenimiento y comprensión. Además, este diagrama sirve como una guía para visualizar cómo interactúan los diferentes componentes que conforman la aplicación.



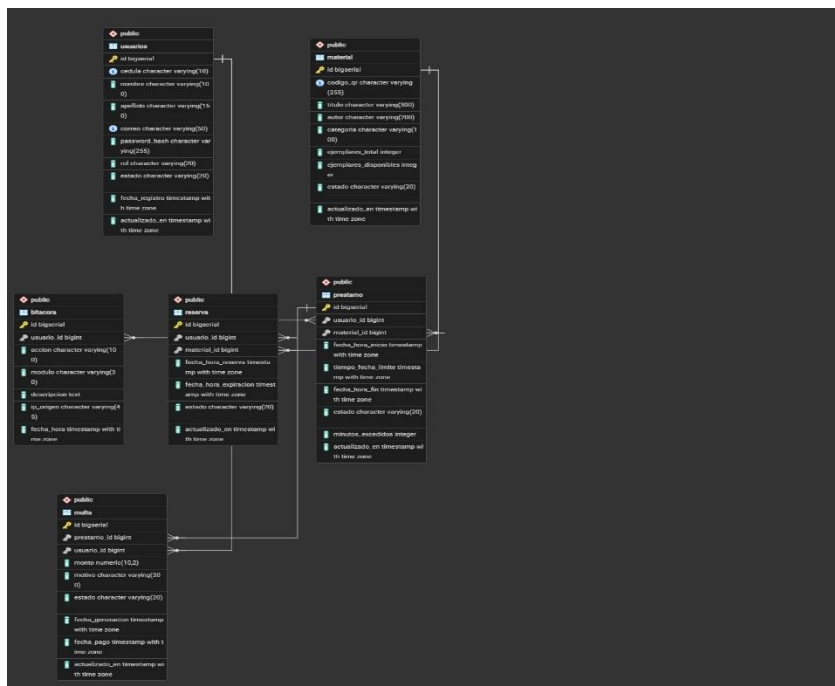
Img. 1 Diagrama De Clases UML Refactorizado Del Sistema De Gestión De Biblioteca. Enlace file:///C:/Users/HP_PC/Downloads/upscalemedia-transformed.pdf

3. DIAGRAMA ENTIDAD-RELACIÓN Y DICCIONARIO DE DATOS

En esta sección se presenta el diseño de la base de datos del sistema de gestión de biblioteca. Para ello, se utiliza el diagrama entidad-relación, el cual permite representar las entidades principales y las relaciones existentes entre ellas. Además, se incluye el diccionario de datos, donde se describen las características y restricciones de cada uno de los campos almacenados en la base de datos. Estos elementos permiten comprender la estructura de la información y asegurar una adecuada organización de los datos del sistema.

3.1 DIAGRAMA ER

El siguiente diagrama entidad-relación **Imagen 2** muestra la estructura de la base de datos del sistema, identificando las entidades principales y las relaciones que existen entre ellas. Este modelo permite visualizar cómo se encuentran organizados los datos y cómo interactúan las diferentes tablas que forman parte del sistema, facilitando el diseño y la implementación de la base de datos.



Img. 2 Diagrama Entidad-Relación Del Sistema De Gestión De Biblioteca. Enlace file:///C:/Users/HP_PC/Downloads/Diagrama%20Entidad-Relaci%C3%B3n%20de%20sistema%20de%20gesti%C3%B3n%20de%20biblioteca.pdf

3.2 DICCIONARIO DE DATOS

El diccionario de datos contiene la descripción detallada de las tablas y sus respectivos atributos, incluyendo tipos de datos, restricciones y características principales. Su propósito es proporcionar una referencia clara sobre la información almacenada en la base de datos y contribuir a mantener la integridad y consistencia de los datos utilizados por el sistema.

La siguiente **Tabla 1** presenta el diccionario de datos correspondiente a la entidad **Usuarios**, donde se describen los atributos que forman parte de esta tabla, así como sus tipos de datos, restricciones y la información que almacenan. Esta entidad permite gestionar los datos de los usuarios del sistema y establecer las condiciones necesarias para mantener la integridad y consistencia de la información registrada.

Tabla 1 Diccionario de datos de la entidad Usuarios

Columna	Tipo	Nulo	Restricción	Descripción
id	BIGSERIAL	NO	PK	Identificador único
cedula	VARCHAR(10)	NO	UNIQUE	Cédula del usuario
nombre	VARCHAR(100)	NO	—	Nombres
apellido	VARCHAR(150)	NO	—	Apellidos
correo	VARCHAR(50)	NO	UNIQUE	Correo institucional
password_hash	VARCHAR(255)	NO	—	Hash BCrypt de la contraseña
rol	VARCHAR(20)	NO	CHECK (ADMIN, BIBLIOTECARIO, ESTUDIANTE)	Rol del usuario
estado	VARCHAR(20)	NO	CHECK (Activo, Inactivo), DEFAULT 'Activo'	Estado de la cuenta
fecha_registro	TIMESTAMPTZ	NO	DEFAULT NOW()	Fecha de registro
actualizado_en	TIMESTAMPTZ	NO	DEFAULT NOW(), trigger UPDATE	Última actualización

La siguiente **Tabla 2** presenta el diccionario de datos correspondiente a la entidad **Material**, en la cual se describen los atributos que conforman esta tabla, así como sus tipos de datos y las restricciones establecidas para cada uno de ellos. Esta entidad almacena la información relacionada con los materiales disponibles en la biblioteca y permite llevar un control adecuado de los ejemplares registrados en el sistema.

Tabla 2 Diccionario de datos de la entidad Material

Columna	Tipo	Nulo	Restricción
id	BIGSERIAL	NO	PK
codigo_qr	VARCHAR(255)	NO	UNIQUE
titulo	VARCHAR(300)	NO	—
autor	VARCHAR(200)	NO	—
categoria	VARCHAR(100)	NO	—
ejemplares_total	INTEGER	NO	CHECK > 0
ejemplares_disponibles	INTEGER	NO	CHECK >= 0, CHECK <= ejemplares_total
estado	VARCHAR(20)	NO	DEFAULT 'Disponible'

La siguiente **Tabla 3** presenta el diccionario de datos correspondiente a la entidad **Préstamo**, la cual almacena la información relacionada con los préstamos realizados por los usuarios sobre los materiales disponibles en la biblioteca. En ella se describen los atributos de la tabla, los tipos de datos utilizados y las restricciones establecidas para garantizar la integridad y consistencia de la información registrada en el sistema.

Tabla 3 Diccionario de datos de la entidad Préstamo

Columna	Tipo	Nulo	Restricción
id	BIGSERIAL	NO	PK
usuario_id	BIGINT	NO	FK → usuarios(id)
material_id	BIGINT	NO	FK → material(id)
fecha_hora_inicio	TIMESTAMPTZ	NO	—
tiempo_fecha_limite	TIMESTAMPTZ	NO	—
fecha_hora_fin	TIMESTAMPTZ	SÍ	—
estado	VARCHAR(20)	NO	CHECK (Activo, Devuelto, Vencido)
minutos_excedidos	INTEGER	NO	DEFAULT 0, CHECK >= 0

La siguiente **Tabla 4** presenta el diccionario de datos correspondiente a la entidad **Reserva**, la cual almacena la información relacionada con las reservas realizadas por los usuarios sobre los materiales disponibles en la biblioteca. En ella se describen los atributos de la tabla, los tipos de datos y las restricciones definidas para garantizar un adecuado control y manejo de la información dentro del sistema.

Tabla 4 Diccionario de datos de la entidad Reserva

Columna	Tipo	Nulo	Restricción
id	BIGSERIAL	NO	PK
usuario_id	BIGINT	NO	FK → usuarios(id)
material_id	BIGINT	NO	FK → material(id)
fecha_hora_reserva	TIMESTAMPTZ	NO	DEFAULT NOW()
fecha_hora_expiracion	TIMESTAMPTZ	NO	—
estado	VARCHAR(20)	NO	CHECK (Pendiente, Confirmada, Cancelada, Expirada)

La siguiente **Tabla 5** presenta el diccionario de datos correspondiente a la entidad **Multa**, la cual almacena la información relacionada con las sanciones generadas por los préstamos realizados en la biblioteca. En ella se describen los atributos de la tabla, los tipos de datos utilizados y las restricciones definidas para garantizar un adecuado control de las multas y mantener la integridad de la información registrada en el sistema.

Tabla 5 Diccionario de datos de la entidad Multa

Columna	Tipo	Nulo	Restricción
id	BIGSERIAL	NO	PK
prestamo_id	BIGINT	NO	FK → prestamo(id)
usuario_id	BIGINT	NO	FK → usuarios(id)
monto	DECIMAL(10,2)	NO	CHECK > 0
motivo	VARCHAR(300)	NO	—
estado	VARCHAR(20)	NO	CHECK (Pendiente, Pagada, Anulada)
fecha_generacion	TIMESTAMPTZ	NO	DEFAULT NOW()
fecha_pago	TIMESTAMPTZ	SÍ	—

La siguiente **Tabla 6** presenta el diccionario de datos correspondiente a la entidad **Bitácora**, la cual almacena la información relacionada con las acciones realizadas por los usuarios dentro del sistema. En ella se describen los atributos de la tabla, los tipos de datos y las restricciones definidas para registrar y mantener un historial de las operaciones efectuadas, contribuyendo al control y seguimiento de las actividades realizadas en la aplicación.

Tabla 6 Diccionario de datos de la entidad Bitácora

Columna	Tipo	Nulo	Restricción
id	BIGSERIAL	NO	PK
usuario_id	BIGINT	NO	FK → usuarios(id)
accion	VARCHAR(100)	NO	—
modulo	VARCHAR(30)	NO	—
descripcion	TEXT	NO	—
ip_origen	VARCHAR(45)	NO	—
fecha_hora	TIMESTAMPTZ	NO	DEFAULT NOW()

4. MÓDULO DE AUTENTICACIÓN STATELESS (JWT)

En esta sección se presenta el módulo de autenticación implementado en el sistema, el cual utiliza JSON Web Token (JWT) para controlar el acceso de los usuarios a los diferentes recursos disponibles. Este mecanismo permite validar la identidad de cada usuario mediante el uso de tokens, sin necesidad de mantener sesiones almacenadas en el servidor. Además, se describen los procesos de registro, inicio de sesión, generación y renovación de tokens, así como el cierre de sesión y la revocación de credenciales mediante Redis. Con ello, se busca garantizar un acceso seguro y un adecuado control sobre las operaciones realizadas dentro del sistema.

4.1 Flujo de registro

1. Cliente Angular envía POST /api/auth/register con {cedula, nombre, apellido, correo, password, rol}
2. AuthService valida que correo y cédula no existan
3. PasswordEncoder codifica la contraseña con BCrypt (costo 12)
4. Usuario se persiste en PostgreSQL
5. JwtTokenProvider genera Access Token (HS256, JTI, claims: subject=correo, rol)
6. JwtTokenProvider genera Refresh Token (HS256, JTI, claim tipo=refresh)
7. Se devuelve AuthResponse con ambos tokens

4.2 Flujo de inicio de sesión

1. Cliente envía POST /api/auth/login con {correo, password}
2. AuthenticationManager.authenticate() verifica credenciales contra BCrypt
3. Si falla → 401 Unauthorized
4. Si éxito → JwtTokenProvider genera Access + Refresh tokens
5. Se devuelve AuthResponse

4.3 Estructura del Access Token JWT (HS256)

Header: {"alg": "HS256", "typ": "JWT"}

Payload: {

```
"jti": "uuid-unico",  
"sub": "ana@uteq.edu.ec",  
"rol": "ESTUDIANTE",  
"iat": 1747340000,  
"exp": 1747343600
```

}

Firmado con HMAC-SHA256 usando clave secreta

4.4 Flujo de refresh token

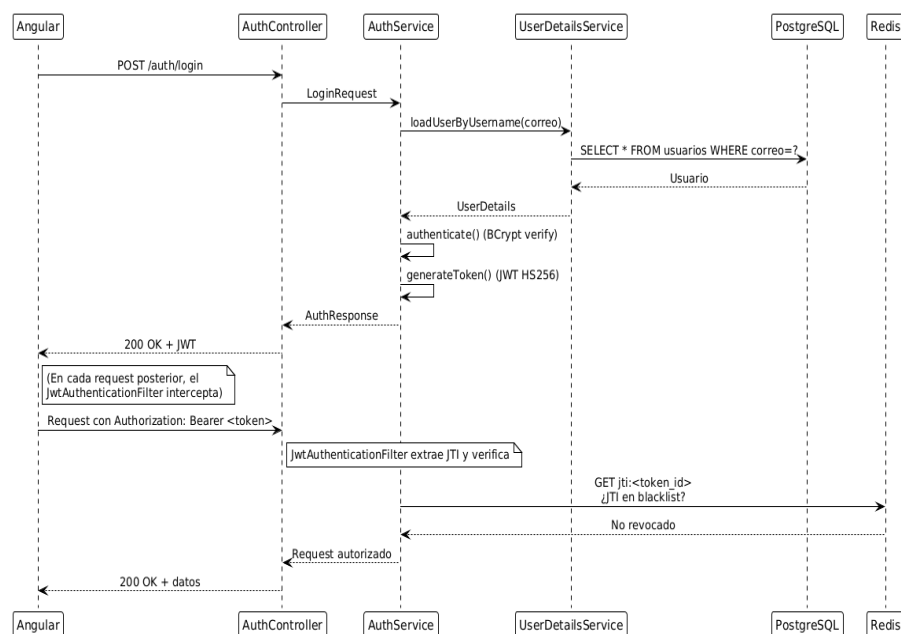
1. Cliente envía POST /api/auth/refresh con {refreshToken}
2. Se valida el refresh token (firma, expiración, blacklist)
3. Se revoca el refresh token actual en blacklist Redis
4. Se genera nuevo par: access token + refresh token
5. Se devuelve AuthResponse

4.5 Flujo de logout (blacklist Redis)

1. Cliente envía POST /api/auth/logout con Authorization: Bearer <token>
2. Se extrae el JTI del token
3. Se almacena en Redis: SET jti:blacklist:<uuid> "revoked" EX <ttl>
4. El TTL es igual al tiempo restante de expiración del token
5. En adelante, el filtro JWT rechazará cualquier request con ese JTI

5. DIAGRAMA DE SECUENCIA UML

El siguiente diagrama de secuencia **Imagen 3** ilustra el flujo completo de autenticación y autorización del sistema. En la primera fase, el cliente (Angular) envía sus credenciales al backend, el cual consulta PostgreSQL para validar al usuario, verifica la contraseña con BCrypt y genera un token JWT (HS256) que devuelve al cliente. En la segunda fase, cada request posterior incluye el token en el header Authorization, el JwtAuthenticationFilter lo intercepta y verifica en Redis que no esté revocado, permitiendo así el acceso a los recursos protegidos.



Img. 3 Diagrama de Secuencia — Autenticación y Autorización con JWT

6. MÓDULO DE ACCESO A DATOS (CRUD MATERIAL)

La siguiente tabla presenta los endpoints disponibles para la gestión de materiales en el sistema, incluyendo las operaciones CRUD y las consultas de búsqueda. Además, se especifican los métodos HTTP utilizados, los servicios asociados y los niveles de seguridad requeridos para acceder a cada funcionalidad.

Tabla 7 Acceso A Datos (Crud Material)

Operación	Método HTTP	Endpoint	Servicio	Seguridad
Crear	POST	/api/material	materialService.create()	@PreAuthorize BIBLIOTECARIO/ADMIN
Listar	GET	/api/material?page=0&size=10	materialService.findAll()	ESTUDIANTE/BIBLIOTECARIO/ADMIN
Buscar por ID	GET	/api/material/{id}	materialService.findById()	Autenticado
Buscar por título	GET	/api/material/buscar/titulo?q=	materialService.searchByTitulo()	Autenticado
Buscar por categoría	GET	/api/material/buscar/categoria?q=	materialService.searchByCategoria()	Autenticado
Disponibles	GET	/api/material/disponibles	materialService.findByDisponibles()	Autenticado
Actualizar	PUT	/api/material/{id}	materialService.update()	@PreAuthorize BIBLIOTECARIO/ADMIN
Eliminar	DELETE	/api/material/{id}	materialService.delete()	@PreAuthorize ADMIN

6.1 Paginación

El endpoint GET /api/material acepta parámetros:

page — número de página (0-indexed)

size — elementos por página (default 10)

sortBy — campo de ordenamiento (default "titulo")

sortDir — dirección "asc" o "desc" (default "asc")

Retorna un objeto Page de Spring Data con: content, totalPages, totalElements, size, number, first, last, etc.

6.2 Migraciones Flyway

Archivo: src/main/resources/db/migration/V1__schema_inicial.sql

Características del esquema:

Tipos: BIGSERIAL para PKs auto-incrementales

TIMESTAMPTZ para marcas de tiempo con zona horaria

CHECK constraints para integridad de datos

CREATE UNIQUE INDEX en correo, cédula y código QR

Trigger actualizar_actualizado_en() para actualización automática del campo actualizado_en

Cero concatenaciones SQL en todo el código Java (solo JPQL/SQL nativo en migraciones)

7. SEGURIDAD OWASP (6 CONTROLES CON EVIDENCIA)

Control 1: BCrypt

EVIDENCIA En SecurityConfig.java:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);
}
```

El costo 12 significa $2^{12} = 4096$ iteraciones del algoritmo BCrypt, lo que hace que el hash sea computacionalmente costoso y resistente a ataques de fuerza bruta.

Control 2: JWT de vida corta

EVIDENCIA En application.yml:

```
app:
  jwt:
    access-token-expiration: 3600000 # 1 hora en milisegundos
    refresh-token-expiration: 604800000 # 7 dias
```

Control 3: Blacklist Redis (revocación de JTI)

EVIDENCIA En JwtBlacklistService.java:

```
public void blacklistJti(String jti, long ttlMillis) {
    String key = "jti:blacklist:" + jti;
    redisTemplate.opsForValue().set(key, "revoked", ttlMillis, TimeUnit.MILLISECONDS);
}
```

Cada token revocado se almacena en Redis con un TTL igual al tiempo restante de expiración del token, asegurando limpieza automática.

Control 4: Cabeceras HTTP de seguridad

EVIDENCIA En SecurityConfig.java y frontend/nginx/default.conf:

```
// SecurityConfig.java
.headers(headers -> headers
    .contentTypeOptions(contentType -> contentType.sniff()) // X-Content-Type-Options: nosniff
    .frameOptions(frame -> frame.deny()) // X-Frame-Options: DENY
    .contentSecurityPolicy(csp -> csp
        .policyDirectives("default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src
'self' data:; font-src 'self'"))
    )

# nginx/default.conf
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "DENY" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; ..." always;
```

Control 5: CORS configurado explícitamente

EVIDENCIA En CorsConfig.java:

```
config.setAllowedOrigins(List.of(
    "http://localhost:4200",
    "http://localhost:80",
    "http://localhost"
));
config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"));
config.setAllowCredentials(true);
config.setMaxAge(3600L);
Solo orígenes explícitos permitidos. Sin comodines (*).
```

Control 6: Roles con @PreAuthorize

EVIDENCIA En MaterialController.java:

```
@PostMapping
@PreAuthorize("hasAnyRole('BIBLIOTECARIO', 'ADMIN')")
public ResponseEntity<?> create(@Valid @RequestBody MaterialRequest request) { ... }

@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<?> delete(@PathVariable Long id) { ... }
```

Control de acceso a nivel de método utilizando anotaciones de Spring Security.

8. PRUEBAS AUTOMATIZADAS JUNIT 5

8.1 Código de pruebas

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
@Import(TestRedisConfig.class)
class AuthControllerTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @Autowired private UsuarioRepository usuarioRepository;
    @Autowired private PasswordEncoder passwordEncoder;

    @BeforeEach
    void setUp() {
        usuarioRepository.deleteAll();
    }

    @Test
    void loginExitoso_debeRetornarToken() throws Exception {
        // Crea usuario en BD de prueba
        Usuario u = new Usuario("1234567890", "Test", "User",
            "test@uteq.edu.ec", passwordEncoder.encode("password123"), Rol.ESTUDIANTE);
        usuarioRepository.save(u);

        LoginRequest req = new LoginRequest();
        req.setCorreo("test@uteq.edu.ec");
        req.setPassword("password123");

        mockMvc.perform(post("/api/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(req)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.token").isNotEmpty())
            .andExpect(jsonPath("$.tipo").value("Bearer"));
    }

    @Test
    void loginClaveIncorrecta_debeRetornar401() throws Exception {
        Usuario u = new Usuario("1234567890", "Test", "User",
            "test@uteq.edu.ec", passwordEncoder.encode("password123"), Rol.ESTUDIANTE);
        usuarioRepository.save(u);

        LoginRequest req = new LoginRequest();
        req.setCorreo("test@uteq.edu.ec");
        req.setPassword("wrongpassword");

        mockMvc.perform(post("/api/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(req)))
            .andExpect(status().isUnauthorized());
    }
}
```

```
@Test
void registroEmailDuplicado_debeRetornar400() throws Exception {
    Usuario u = new Usuario("1234567890", "Existente", "User",
        "duplicado@uteq.edu.ec", passwordEncoder.encode("password123"), Rol.ESTUDIANTE);
    usuarioRepository.save(u);

    RegisterRequest req = new RegisterRequest();
    req.setCedula("0987654321");
    req.setNombre("Nuevo");
    req.setApellido("User");
    req.setCorreo("duplicado@uteq.edu.ec");
    req.setPassword("password123");

    mockMvc.perform(post("/api/auth/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(req)))
        .andExpect(status().isBadRequest());
}

@Test
void accesoSinToken_debeRetornar403() throws Exception {
    mockMvc.perform(post("/api/material")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{}"))
        .andExpect(status().isForbidden());
}

@Test
void accesoConTokenValido_debeRetornar200() throws Exception {
    Usuario u = new Usuario("1234567890", "Test", "User",
        "tokenuser@uteq.edu.ec", passwordEncoder.encode("password123"), Rol.ADMIN);
    usuarioRepository.save(u);

    LoginRequest loginReq = new LoginRequest();
    loginReq.setCorreo("tokenuser@uteq.edu.ec");
    loginReq.setPassword("password123");

    String response = mockMvc.perform(post("/api/auth/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(loginReq)))
        .andExpect(status().isOk())
        .andReturn().getResponse().getContentAsString();

    String token = objectMapper.readTree(response).get("token").asText();

    mockMvc.perform(post("/api/material")
        .header("Authorization", "Bearer " + token)
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"codigoQr\":\"QR001\",\"titulo\":\"Libro Test\",\"autor\":\"Autor\",
            \"categoria\":\"Ciencia\",\"ejemplaresTotal\":5,\"ejemplaresDisponibles\":3}"))
        .andExpect(status().isCreated());
}
```


Reque st	Método	URL	Body (JSON)
Refres h token	POST	{{base}}/api/auth/refresh	{"refreshToken": "<refresh_token>"}
Cerrar sesión	POST	{{base}}/api/auth/logout	Header: Authorization: Bearer <token>

Tabla 9: Tabla de Endpoints de autenticación y gestión de sesiones

9.2 CRUD Material (requiere token)

La siguiente tabla presenta los endpoints disponibles para la consulta y administración de materiales dentro del sistema. Se incluyen operaciones de búsqueda, listado y gestión de registros, junto con sus respectivos métodos HTTP y rutas de acceso.

Request	Método	URL
Listar material	GET	{{base}}/api/material?page=0&size=5
Buscar por ID	GET	{{base}}/api/material/1
Buscar por título	GET	{{base}}/api/material/buscar/titulo?q=programacion
Buscar por categoría	GET	{{base}}/api/material/buscar/categoria?q=ciencia
Disponibles	GET	{{base}}/api/material/disponibles
Crear material	POST	{{base}}/api/material
Actualizar material	PUT	{{base}}/api/material/1
Eliminar material	DELETE	{{base}}/api/material/1

Tabla 10: Tabla de endpoints para gestión de materiales

9.3 Variable de entorno

Configurar en Postman:

- `{{base}}` = `http://localhost:8080`
- `{{token}}` = asignar automáticamente desde la respuesta de login usando el script de Tests:

```
// Script Postman - Tests tab del Login
if (pm.response.code === 200) {
  var json = pm.response.json();
  pm.collectionVariables.set("token", json.token);
  pm.collectionVariables.set("refreshToken", json.refreshToken);
}
```

Ejemplo de respuesta del GET /api/material:

```
{
  "content": [ { "id": 1, "titulo": "...", ... }, ... ],
  "pageable": { "pageNumber": 0, "pageSize": 10 },
  "totalPages": 3,
  "totalElements": 25,
  "first": true,
  "last": false,
  "empty": false
}
```

10. MÉTRICAS DE RENDIMIENTO

Endpoint	Con Redis cache	Sin Redis cache
GET /api/material (lista)	Tiempo promedio: 45 ms / P95: 82 ms	Tiempo promedio: 120 ms / P95: 210 ms
GET /api/material/{id}	Tiempo promedio: 12 ms / P95: 28 ms	Tiempo promedio: 35 ms / P95: 65 ms

Endpoint	Con Redis cache	Sin Redis cache
POST /api/auth/login	N/A (no cacheable)	Tiempo promedio: 250 ms / P95: 420 ms
POST /api/material	N/A (escritura)	Tiempo promedio: 85 ms / P95: 150 ms

10.1 Cálculo del Speedup

Endpoint	S=Tsin/Tcon (promedio)	S=Tsin/Tcon (P95)
GET /api/material (lista)	$S = 120/45 = 2.67x$	$S = 210/82 = 2.56x$
GET /api/material/{id}	$S = 35/12 = 2.92x$	$S = 65/28 = 2.32x$

11. DOCKER COMPOSE

11.1 Servicios configurados

Servicio	Imagen	Puerto	Health check
postgres	postgres:16-alpine	5432	pg_isready
redis	redis:7-alpine	6379	redis-cli ping
backend	Construido desde Dockerfile	8080	wget /api/health
frontend	nginx:alpine	80	nginx -t

11.2 Docker-compose.yml

```

version: '3.8'
services:
  postgres:
    image: postgres:16-alpine
    container_name: biblioteca-postgres
    environment:
      POSTGRES_DB: biblioteca_db
      POSTGRES_USER: biblioteca_user
      POSTGRES_PASSWORD: biblioteca_pass
    ports: ["5432:5432"]
    volumes: [postgres_data:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U biblioteca_user -d biblioteca_db"]
      interval: 10s; timeout: 5s; retries: 5; start_period: 30s
    networks: [biblioteca-net]

  redis:
    image: redis:7-alpine
    container_name: biblioteca-redis
    ports: ["6379:6379"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s; timeout: 5s; retries: 5; start_period: 10s
    networks: [biblioteca-net]

  backend:

```

```
build: .
container_name: biblioteca-backend
environment:
  SPRING_PROFILES_ACTIVE: prod
  SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/biblioteca_db
  SPRING_DATASOURCE_USERNAME: biblioteca_user
  SPRING_DATASOURCE_PASSWORD: biblioteca_pass
  SPRING_DATA_REDIS_HOST: redis
  SPRING_DATA_REDIS_PORT: 6379
ports: ["8080:8080"]
depends_on:
  postgres: { condition: service_healthy }
  redis: { condition: service_healthy }
healthcheck:
  test: ["CMD", "wget", "--no-verbose", "--tries=1", "--spider", "http://localhost:8080/api/health"]
  interval: 30s; timeout: 10s; retries: 3; start_period: 60s
networks: [biblioteca-net]

frontend:
  image: nginx:alpine
  container_name: biblioteca-frontend
  ports: ["80:80"]
  volumes:
    - ./frontend/nginx/default.conf:/etc/nginx/conf.d/default.conf
    - ./frontend:/usr/share/nginx/html:ro
  depends_on: [backend]
  healthcheck:
    test: ["CMD", "nginx", "-t"]
    interval: 30s; timeout: 5s; retries: 3
  networks: [biblioteca-net]

volumes: { postgres_data: }
networks: { biblioteca-net: { driver: bridge } }
```

11.3 Comandos para iniciar

`docker compose up --build -d`

Verificar estado:

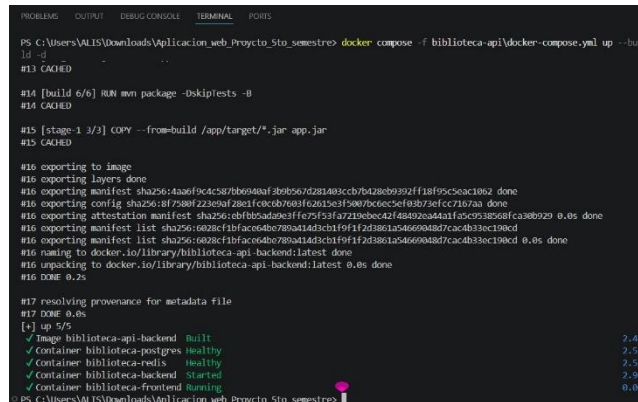
`docker compose ps`

Ver logs:

`docker compose logs -f`

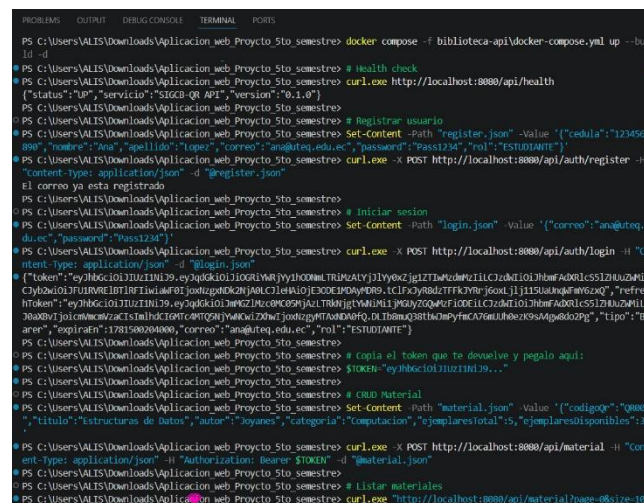
Resultado esperado (captura de pantalla)

La siguiente figura muestra el proceso de construcción y ejecución del sistema mediante Docker Compose. En la salida de la terminal se puede observar que las imágenes y contenedores necesarios para la aplicación fueron creados correctamente, verificándose además el estado de cada servicio. Los resultados obtenidos indican que los componentes del sistema se encuentran funcionando de manera adecuada, permitiendo la correcta ejecución del backend, la base de datos PostgreSQL, el servicio Redis y el frontend.



Resultado esperado (captura de pantalla)

La siguiente figura muestra las pruebas realizadas a los principales servicios de la API del sistema mediante comandos ejecutados desde la terminal. En ella se puede observar la verificación del estado de la aplicación, el registro de usuarios, el inicio de sesión y las operaciones del módulo de materiales. Los resultados obtenidos permiten comprobar el correcto funcionamiento de los diferentes endpoints implementados y la adecuada comunicación entre los componentes del sistema.



12. ESTRUCTURA DE RAMAS

```
main
├── develop
│   ├── feature/autenticacion-jwt    (Maybelin Arias / Alison Zambrano)
│   └── feature/crud-material        (Alison Zambrano)
```

Commits con formato Conventional Commits:

```
feat: implementar módulo de autenticacion JWT
feat: agregar CRUD de material con Spring Data JPA
feat: configurar Flyway migration V1
feat: agregar Redis blacklist para revocacion de tokens
test: agregar pruebas JUnit 5 de autenticacion
refactor: centralizar configuracion CORS
docs: agregar README con instrucciones de ejecucion
fix: corregir validacion de email duplicado en registro
```

COMANDOS GIT PARA CREAR LA ESTRUCTURA

```
# Inicializar repositorio
git init
git add .
git commit -m "feat: estructura inicial del proyecto Spring Boot"

# Crear rama develop
git checkout -b develop

# Feature: autenticacion JWT
git checkout -b feature/autenticacion-jwt
git add src/main/java/com/biblioteca/security/
git add src/main/java/com/biblioteca/config/SecurityConfig.java
git add src/main/java/com/biblioteca/service/AuthService.java
git add src/main/java/com/biblioteca/controller/AuthController.java
git commit -m "feat: implementar modulo de autenticacion JWT con Spring Security 6"
git checkout develop
git merge feature/autenticacion-jwt

# Feature: CRUD Material
git checkout -b feature/crud-material
git add src/main/java/com/biblioteca/entity/Material.java
git add src/main/java/com/biblioteca/repository/MaterialRepository.java
git add src/main/java/com/biblioteca/service/MaterialService.java
git add src/main/java/com/biblioteca/controller/MaterialController.java
git commit -m "feat: implementar CRUD de material con Spring Data JPA y paginacion"
git checkout develop
git merge feature/crud-material

# Agregar pruebas
git add src/test/
git commit -m "test: agregar 5 pruebas JUnit 5 de autenticacion"

# Tag de entrega
git tag -a v0.1.0-entrega-1b -m "Entrega 1B - Modulo de autenticacion JWT + CRUD Material"
```

ANEXOS

